

**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em TypeScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Generics

o que são

- parâmetro de tipo polimórfico
- usado quando o tipo é desconhecido antes do uso
- impõe uma restrição de nível de tipo em vários lugares
- define o tipo "genérico" <T>

exemplos

```
// JS
function filter(array, f) {

    let result = []

    for (let i = 0; i < array.length; i++) {
        let item = array[i]
        if (f(item)) {
            result.push(item)
        }
    }

    return result
}

// TS
type Filter = <T> (
    array: T[],
    f: (item: T) => boolean
) => T[];
```



Generics

o que são

- parâmetro de tipo polimórfico
- usado quando o tipo é desconhecido antes do uso
- impõe uma restrição de nível de tipo em vários lugares
- define o tipo "genérico" <T>

exemplos

```
type Filter = <T>(  
  array: T[],  
  f: (item: T) => boolean  
) => T[];  
  
let filter: Filter = (array, f) => {  
  let result = []  
  
  for (let i = 0; i < array.length; i++) {  
  
    let item = array[i]  
  
    if (f(item)) {  
      result.push(item)  
    }  
  }  
  
  return result  
}
```

Classes

definição

- definidas como em JS
- suporte para:
 - iniciadores de propriedades
 - modificadores de visibilidade
 - abstração
 - interfaces
 - polimorfismo

exemplos

```
class Retangulo {  
  
    constructor(largura: number, altura: number) {  
        this.largura = largura;  
        this.altura = altura;  
    }  
  
    area(): number {  
        return this.largura * this.altura;  
    }  
  
    perimetro(): number {  
        return 2*(this.largura + this.altura);  
    }  
}
```



Classes

definição

- definidas como em JS
- suporte para:
 - **iniciadores de propriedades**
 - modificadores de visibilidade
 - abstração
 - interfaces
 - polimorfismo

exemplos

```
class Retangulo {  
    _largura: number;  
    _altura: number;  
  
    constructor(largura: number, altura: number) {  
        this._largura = largura;  
        this._altura = altura;  
    }  
  
    get largura(): number {  
        return this._largura;  
    }  
  
    get altura(): number {  
        return this._altura;  
    }  
}
```



Classes

definição

- definidas como em JS
- suporte para:
 - iniciadores de propriedades
 - **modificadores de visibilidade**
 - abstração
 - interfaces
 - polimorfismo

exemplos

```
class Retangulo {  
  private _largura: number;  
  private _altura: number;  
  
  constructor(largura: number, altura: number) {  
    this._largura = largura;  
    this._altura = altura;  
  }  
  
  protected area(): number {  
    return this._largura * this._altura;  
  }  
  
  perimetro(): number {  
    return 2*(this._largura + this._altura);  
  }  
}
```



Classes

definição

- definidas como em JS
- suporte para:
 - iniciadores de propriedades
 - **modificadores de visibilidade**
 - abstração
 - interfaces
 - polimorfismo

exemplos

```
class Retangulo {  
  
    constructor(  
        private _largura: number,  
        private _altura: number  
    ) {}  
  
    protected area(): number {  
        return this._largura * this._altura;  
    }  
  
    public perimetro(): number {  
        return 2*(this._largura + this._altura);  
    }  
}
```



Classes

definição

- definidas como em JS
- suporte para:
 - iniciadores de propriedades
 - modificadores de visibilidade
 - **abstração**
 - interfaces
 - polimorfismo

exemplos

```
abstract class Transporte {  
    constructor(public nome: string) {}  
    abstract mover(): void;  
    parar(): void {  
        console.log(`${this.nome} parou.`);  
    }  
}  
  
class Carro extends Transporte {  
    mover(): void {  
        console.log(`${this.nome} está dirigindo.`);  
    }  
}  
  
class Aviao extends Transporte {  
    mover(): void {  
        console.log(`${this.nome} está voando.`);  
    }  
}
```



Classes

definição

- definidas como em JS
- suporte para:
 - iniciadores de propriedades
 - modificadores de visibilidade
 - abstração
 - **interfaces**
 - polimorfismo

exemplos

```
interface Animal {
    readonly nome: string;
    comer(comida: string): void;
    dormir(horas: number): void;
}

interface Felino {
    miar(): void;
}

class Gato implements Animal, Felino {
    nome: string = 'Bichano';
    comer(comida: string) {
        console.log('Comi ', comida, '. Mmm!');
    };
    dormir(horas: number) {
        console.log('Dormi por', horas, 'horas');
    };
    miar() { console.log('Miau!'); };
}
```



Classes

definição

- definidas como em JS
- suporte para:
 - iniciadores de propriedades
 - modificadores de visibilidade
 - abstração
 - interfaces
 - **polimorfismo**

exemplos

```
interface Queue<T> {  
    enqueue(item: T): void;  
    dequeue(): T | undefined;  
    size(): number;  
}  
  
class ArrayQueue<T> implements Queue<T> {  
    private items: T[] = [];  
    enqueue(item: T): void {  
        this.items.push(item);  
    }  
    dequeue(): T | undefined {  
        return this.items.shift();  
    }  
    size(): number {  
        return this.items.length;  
    }  
}
```



Classes

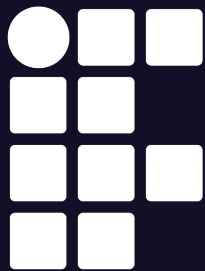
definição

- definidas como em JS
- suporte para:
 - iniciadores de propriedades
 - modificadores de visibilidade
 - abstração
 - interfaces
 - **polimorfismo**
- são estruturalmente tipadas
 - classes com a mesma forma são compatíveis
 - **exceção** classes com campos privados (private) ou protegidos (protected)

exemplos

```
class Zebra {  
    trotar() { console.log("A zebra está trotando!"); }  
}  
  
class Poodle {  
    trotar() { console.log("O poodle está trotando!"); }  
}  
  
function passear(animal: Zebra) {  
    animal.trotar();  
}  
  
let zebra = new Zebra();  
passear(zebra);  
  
let poodle = new Poodle();  
passear(poodle);
```





**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em TypeScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas